DEPARTMENT OF ELECTRICAL ENGINEERING

COLLEGE OF E&ME, NUST,
RAWALPINDI

FINAL PROJECT

Ultrasonic Distance Measurement and Barrier Control System

SUBMITTED TO:

Course Instructor: Dr Hassan Khalid
Lab Instructor: Ma'am Moazama

SUBMITTED BY:

M. Shahryar Ameer (CMD: 520657)
Seemal Rizwan(CMD: 505272)

DE- 46 (A)
Dept : Electrical Engineering

Submission Date:05-05-2026

TABLE OF CONTENTS:

Abstract:

This project presents an Ultrasonic Distance Measurement and Barrier Control System designed to enhance parking assistance and automated gate control. The system employs an ultrasonic sensor (HC-SR04) interfaced with an ATmega microcontroller to measure the distance to nearby obstacles. The measured distance is displayed on a 16x2 character LCD, providing real-time feedback. When the distance to an obstacle is detected to be less than or equal to 10 cm, the system triggers a barrier mechanism to raise, simulating the opening of a gate. The project demonstrates effective integration of sensor data acquisition, real-time processing, and actuation control, offering a practical solution for applications in automated parking systems and access control. The system is tested for reliability and responsiveness, showing accurate distance measurements and timely barrier activation. Future enhancements may include additional sensors, remote monitoring capabilities, and advanced barrier control mechanisms.

1. Introduction:

The Ultrasonic Distance Measurement and Barrier Control System is designed to assist in parking by measuring the distance to an obstacle and displaying it on an LCD. Additionally, it raises a barrier when the distance to the obstacle is less than or equal to 10 cm. This system can be used in parking lots or garages to automate gate control based on the proximity of a vehicle.

2. Objectives:

- To measure the distance to an obstacle using an ultrasonic sensor.
- To display the measured distance on an LCD.
- To raise a barrier when the measured distance is less than or equal to 10 cm.

3. Hardware Components:

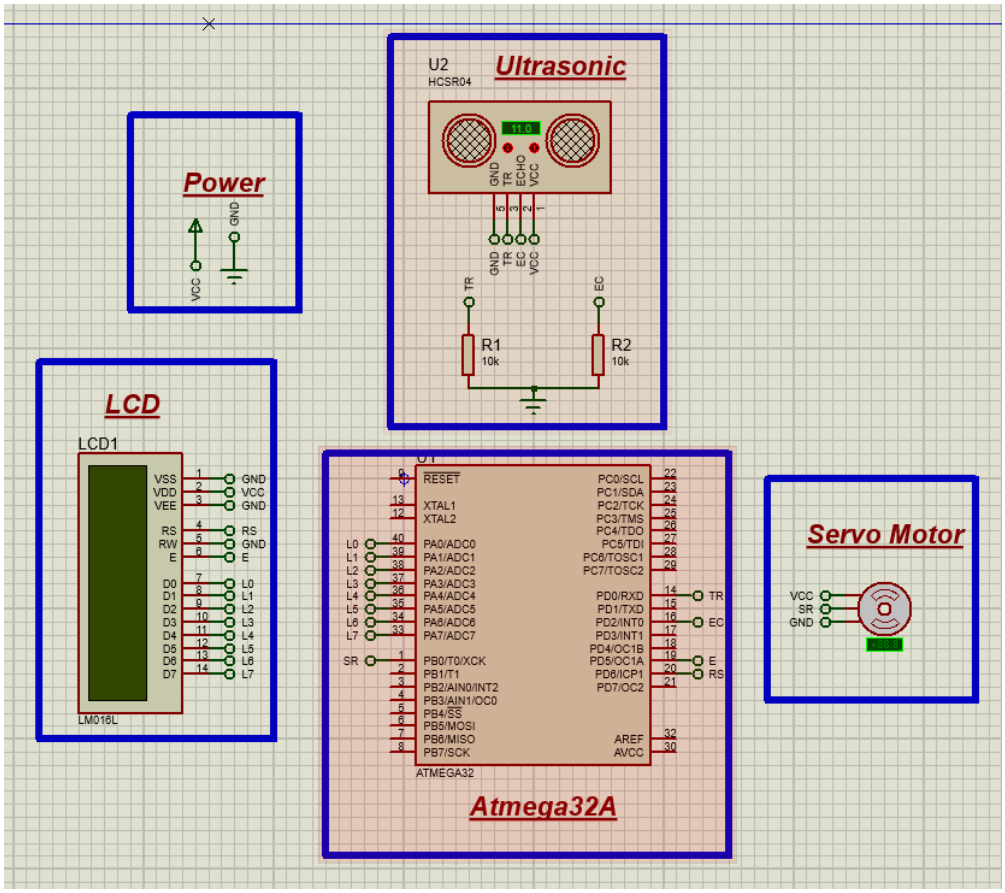
- Microcontroller: ATmega32A
- Ultrasonic Sensor: HC-SR04
- LCD Display: 16x2 Character LCD
- Barrier Mechanism: Servo Motor
- Miscellaneous: Bread Board, Connecting Wires, Development Board

4. Software Components:

- Development Environment: MPLABxIDE XC8 Compiler
- Programming Language: C
- Libraries: AVR Standard Libraries (avr/io.h, avr/interrupt.h, util/delay.h, stdlib.h)

5. Circuit Design:

The system is built around an ATmega32A microcontroller. The ultrasonic sensor (HC-SR04) is connected to the microcontroller to measure distances. The LCD is used to display the distance, and a servo motor is used to control the barrier.



6. System Design & Functionality:

The system works as follows:

- The microcontroller sends a trigger signal to the ultrasonic sensor.
- The sensor sends out an ultrasonic pulse and waits for it to bounce back.
- The echo pin of the sensor generates a pulse width that is proportional to the distance to the obstacle.
- The microcontroller measures this pulse width using an external interrupt.
- The distance is calculated from the pulse width and displayed on the LCD.
- If the distance is less than or equal to 10 cm, the barrier is raised.

7. Code Explaination:

Pin Configuration of Atmega32A and Peripherals attached to it are as follow:

- LCD Enable Pin ->PD5
- LCD Register Select Pin ->PD6
- LCD Data Pins -> PA0-PA7
- Ultrasonic Trigger Pin ->PD0
- Ultrasonic Echo Pin ->PD2
- Servo Motor PWM Pin ->PB0

Code:

```
#include <avr/io.h>

#include <avr/interrupt.h>
```

```
#define F_CPU 1000000UL
#include <util/delay.h>
#include <stdlib.h>

#define enable_pin       PD5
#define register_select_pin PD6

volatile int pulse = 0;
volatile int i = 0;

void send_a_command(unsigned char command);
void send_a_character(unsigned char character);
void send_a_string(const char *string_of_characters);
void barrier_up(void);
void barrier_down(void);
int main(void) {
   DDRA = 0xFF;
   DDRB = 0x01;
   DDRD = 0b11111011;

   PORTB=0;

   _delay_ms(50);

   GICR |= (1 << INT0);
   MCUCR |= (1 << ISC00);

   TCCR1A = 0;

   int16_t COUNTA = 0;
   char SHOWA[16];

   send_a_command(0x01);
   _delay_ms(2);
   send_a_command(0x38);
```

```
    _delay_us(50);
    send_a_command(0b00001111);
    _delay_us(50);

    sei();

    while (1) {
        PORTD |= (1 << PIND0);
        _delay_us(15);
        PORTD &= ~(1 << PIND0);

        COUNTA = pulse / 58;

        send_a_string("EMENENT PARKING");
        send_a_command(0x80 + 0x40 + 0);
        send_a_string("DISTANCE=");
        itoa(COUNTA, SHOWA, 10);
        send_a_string(SHOWA);
        send_a_string("cm    ");
        send_a_command(0x80 + 0);
        if(COUNTA<=10){
            barrier_up();
    }
        else{
            barrier_down();
        }
    }
    return 0;
}

ISR(INT0_vect) {
    if (i == 1) {
        TCCR1B = 0;
        pulse = TCNT1;
        TCNT1 = 0;
```

```
        i = 0;
    }

    if (i == 0) {
        TCCR1B |= (1 << CS10);
        i = 1;
    }
}

void itoa(int n, char s[], int radix) {
    int i, sign;

    if ((sign = n) < 0)
        n = -n;

    i = 0;
    do {

        s[i++] = n % radix + '0';
    } while ((n /= radix) > 0);
    if (sign < 0)
        s[i++] = '-';
    s[i] = '\0';

    int length = i;
    for (i = 0; i < length / 2; ++i) {
        char temp = s[i];
        s[i] = s[length - i - 1];
        s[length - i - 1] = temp;
    }
}
void barrier_up(void)
{
        PORTB = 0x01;
        _delay_us(1500);
```

```
        PORTB = 0x00;
        _delay_ms(200);


}
void barrier_down(void){
     PORTB = 0x01;
       _delay_us(1000);
       PORTB = 0x00;
}
void send_a_command(unsigned char command) {
   PORTA = command;
   PORTD &= ~(1 << register_select_pin);
   PORTD |= (1 << enable_pin);
   _delay_ms(8);
   PORTD &= ~(1 << enable_pin);
   PORTA = 0;
}

void send_a_character(unsigned char character) {
   PORTA = character;
   PORTD |= (1 << register_select_pin);
   PORTD |= (1 << enable_pin);
   _delay_ms(8);
   PORTD &= ~(1 << enable_pin);
   PORTA = 0;
}

void send_a_string(const char *string_of_characters) {
   while (*string_of_characters) {
      send_a_character(*string_of_characters++);
   }
}
```

Global Variables and Definitions

- enable_pin and register_select_pin: Constants for LCD control pins.
- pulse: A volatile integer to store the duration of the echo pulse from the ultrasonic sensor.
- i: A volatile integer used as a state variable in the interrupt service routine.

Function: send_a_command(unsigned char command)

This function sends a command to the LCD.

- Sets `PORTA` to the command value.
- Clears the register select pin (RS) to indicate that a command is being sent.
- Toggles the enable pin (EN) to latch the command into the LCD.
- Delays to allow the command to be processed by the LCD.

Function: send_a_character(unsigned char character)

This function sends a character to the LCD.

- Sets `PORTA` to the character value.
- Sets the register select pin (RS) to indicate that data is being sent.
- Toggles the enable pin (EN) to latch the data into the LCD.
- Delays to allow the character to be processed by the LCD.

Function: send_a_string(const char *string_of_characters)

This function sends a string of characters to the LCD.

- Iterates through each character in the string.
- Calls `send_a_character` to send each character to the LCD.

Function: barrier_up(void)

This function raises the barrier.

- Sets `PORTB` to 0x01 to activate the motor.
- Delays for 1500 microseconds to raise the barrier.
- Resets `PORTB` to 0x00 to stop the motor.
- Delays for 200 milliseconds to allow the barrier to stabilize.

Function: barrier_down(void)

This function lowers the barrier.

- Sets `PORTB` to 0x01 to activate the motor.
- Delays for 1000 microseconds to lower the barrier.
- Resets `PORTB` to 0x00 to stop the motor.

Function: itoa(int n, char s[], int radix)

This function converts an integer `n` to a string `s` using the specified `radix` (base).

- Converts the integer to its absolute value if it is negative.
- Fills the string with characters representing the digits of the integer.
- Adds a negative sign if the integer was negative.
- Reverses the string to get the correct representation.

Function: ISR(INT0_vect)

This interrupt service routine handles the external interrupt from the echo pin of the ultrasonic sensor.

- If i is 1, stops the timer, records the pulse width, and resets the timer and state variable.
- If i is 0, starts the timer and sets the state variable.

Function: main(void)

The main function initializes the system and contains the main loop.

- Configures the data direction registers for the ports.
- Initializes PORTB to 0.
- Enables the external interrupt for INT0.
- Configures the interrupt to trigger on any logical change.
- Initializes Timer1.
- Sends initialization commands to the LCD.
- Enables global interrupts.
- Enters an infinite loop where:
- Sends a trigger pulse to the ultrasonic sensor.
- Calculates the distance from the pulse width.
- Displays the distance and relevant messages on the LCD.
- Checks the distance and raises or lowers the barrier accordingly.

8. Testing & Results:

The system was tested with various distances, and the measured distance was displayed on the LCD correctly. The barrier was successfully raised when the distance was less than or equal to 10 cm. The response time of the system was within acceptable limits for real-time operation.

9. Conclusion:

The Ultrasonic Distance Measurement and Barrier Control System successfully measures the distance to an obstacle and controls a barrier based on the measured distance. The system is reliable and can be used in applications such as parking assistance and automated gate control.